

Introduction

A single page application is an application that works in a browser and does not need page reloading during its use.

Examples of single page applications are Gmail, Google Map, Facebook, Github etc.

ReactJS is a javascript library by Facebook used to create single page applications.

Some of the other common javascript libraries/frameworks used to create single page applications are Angular, Vue JS etc. They are also called as client-side frameworks.

Two important things that need to be known while using React are:-

JSX and Virtual DOM

JSX stands for Javascript XML and is a syntax extension of Javascript.

JSX allows us to write HTML in React

Virtual DOM is a virtual representation (a copy) of the actual DOM by JavaScript. It is kept in memory. It is used in web frameworks such as React, VueJS etc. Updating the virtual DOM is comparatively faster than the actual DOM.

Setup ReactJS Environment

Step 1: Install NodeJS

Go to <https://nodejs.org/en/download/> and install the latest version of nodejs

After installing open command prompt and verify the version of nodejs and npm by the following command

```
node -v
npm -v
```

COPY

Step2: Install React

```
npm install -g create-react-app
```

COPY

Create & Run ReactJS application

To create a react app, run the following code

Step 1:

```
npx create-react-app hello-world
```

COPY

To run the application, do the following

Step 1: Get inside the react app

COPY

```
cd hello-world
```

Step 2: Start the sever

COPY

```
npm start
```

Wait for sometime and you can see that a new window will open with the url <http://localhost:3000>
This shows your react app is working correctly.

IDE

We will be using VS Code IDE throughout this tutorial.

Installing VSCode

Step 1: Go to <https://code.visualstudio.com/download>

Step 2: Download the file

Step 3: Double click the file and install the application

Directory Structure

Upon installation, you will have the following files/folders within the project

node_modules folder

.gitignore

package.json

src folder

index.js

app.js

node_modules: This folder will contain all the react js dependencies

.gitignore: this file is used by git to decide which files/folders needs to be ignored during git commit

package.json: This file contains the dependencies and scripts required for the project

src folder: this is the main folder in the project

index.js: This is the file that will be first executed when we run the project

app.js: This is a component that will be loaded under index.js file

Labs:

Open package.json file and try to find out the version of react js that you have installed.

← PREV

NEXT →

← **PREV**

NEXT →

Components:

In React, Components are the building blocks to create the UI (User Interface). Components are reusable pieces of code that can be composed together to form more complex UIs.

There are two types of components in react:

- Functional components
- Class components

Functional Components

Functional components are javascript functions that take in props (short for properties) and return a react element to be rendered. They are stateless and dont have their own internal state.

Example for functional components.

```
function Greeting(props){
  return(
    <div>
      <h1>Hello {props.name} </h1>
    </div>
  )
}
export default Greeting;
```

COPY

Class Components

Class components are ES6 classes. They are stateful components. They have their own state and life cycle methods.

Example for class components.

```
class counter extends React.components {
  constructor(props) {
    super(props);
    this.state = { count : 0};
  }

  render (){
    return(
      <div>
        <h1>{this.state.count}</h1>
      </div>
    )
  }
}
```

COPY

App Component

In React, the App component is the root component of the react application. It is the top-level component that contains all other components.

COPY

```
function App(){
  return (
    <div>.....</div>
  )}
export default app;
```

← PREV

NEXT →

← PREV

NEXT →

Writing Hello World

Replace `app.js` with the following code

```
function App() {
  return (
    <div>
      <h1>Hello World</h1>
    </div>
  );
}
export default App;
```

COPY

The above code will print "Hello world" in the browser

✓

Activity

Try adding a paragraph to the webpage

Try adding a table and an unordered list to the page.

NOTE: `{ }` is used to display a variable

Writing Hello World with variable

In this example, we store "Hello" to a variable called `greeting` and then display it.

Replace `app.js` with the following code

```
function App() {
  let greeting = "Hello World";
  return (
    <div>
      <h1>{greeting}</h1>
    </div>
  );
}
export default App;
```

COPY

The above code will print "Hello World" in the browser.

✓

Activity

Try printing "Welcome to MashupStack. I am Sreenath" using variable.

Writing Hello World with content and variable

In this example, we store "World" to a variable called greeting and then display "Hello World" by using both hardcoded text and variable.

Replace app.js with the following code

```
function App() {
  let greeting = "World";
  return (
    <div>
      <h1>Hello {greeting}</h1>
    </div>
  );
}
export default App;
```

COPY

The above code will print "Hello World" in the browser.

☒

Activity

Try printing "Welcome to MashupStack. I am Sreenath" where the text "Sreenath" comes from a variable.

Writing Hello World with Inline CSS

Replace the App.js with the following code.

```
function App() {
  return (
    <div>
      <h1 style={{color:"red"}}>Hello World</h1>
    </div>
  );
}
export default App;
```

COPY

Writing Hello World with CSS

In this example, we store the css style to a variable called greetingStyle and use it to style the <h1> element.

Replace the app.js with the following code

```
function App() {
  let greetingStyle = {"color":"red"};
  let greeting = "Hello World";
  return (
    <div>
      <h1 style={greetingStyle}>{greeting}</h1>
    </div>
  );
}
```

COPY

The above code will print "Hello World" in the browser in red color.

Writing Hello World with external CSS

In this example, we use CSS from an external file called "App.css" to add the style. You need to first import the App.css file within App.js. Note that this style will only be applied to the files that have imported App.css file

Replace the app.js with the following code

COPY

```
import './App.css';
function App() {
  let greeting = "Hello World";
  return (
    <div>
      <h1 className="greetingStyle">{greeting}</h1>
    </div>
  );
}
```

Replace the App.css with the following code

COPY

```
.greetingStyle{
  color:green;
}
```

The above code will print "Hello World" in green color.

Writing Hello World with Global CSS

In this example, we use CSS from the global stylesheet "index.css" to add the style. The styles in index.css will work in all the files of the project.

Replace the app.js with the following code

COPY

```
function App() {
  let greeting = "Hello World";
  return (
    <div>
      <h1 className="greetingStyle">{greeting}</h1>
    </div>
  );
}
export default App;
```

Replace the index.css with the following code

COPY

```
.greetingStyle{
  color:blue;
}
```

The above code will print "Hello World" in blue color.

☒ Activity

Create example.com

← **PREV**

NEXT →

Adding bootstrap in react

To add Bootstrap CDN in react application, you can follow these steps:
Open your react application in your preferred editor.
In your public/index.html file, add the Bootstrap CDN link inside the head section of your HTML file.

Replace your **App.js** with the following code

App.js

```
import React from 'react';

function App(){
  return(
    <div className='container'>
      <div className='row'>
        <div className='col-md-4 bg-primary mr-2'> column 1</div>
        <div className='col-md-4 bg-secondary' > column 2</div>
      </div>
      ...
    )
}
```

COPY

Printing variable in console

In this example, we will print a variable in the console by adding the following code

```
let greeting = "Hello World";
console.log(greeting);
```

COPY

Replace **App.js** with the following code

```
import './App.css';
function App() {
  let greeting = "Hello World";
  console.log(greeting);
  return (
    <div>
      <h1 className="greetingStyle">{greeting}</h1>
    </div>
  );
}
```

COPY

The above code will print "Hello World" in the browser as well as the console.

Showing an external image

In this example, we will display an image from an external website

Replace **App.js** with the following code

COPY

```
import './App.css';
function App() {
  let greeting = "Hello World";
  return (
    <div>
    <h1 className="greetingStyle">{greeting}</h1>
    
    <h1 className="greetingStyle">{greeting}</h1>
    <img src={image}/>
    </div>
  );
```

The above code will display an image

← PREV

NEXT →

← PREV

NEXT →

Printing a number

Replace the `app.js` with the following code.

```
function App(){
  const id = 1;
  const mark = 92.5;
  return (
    <div>
      <p>ID is : {id} </p>
      <p> mark is : {mark}</p>
    </div>
  )
}
```

COPY

Printing a string

Replace the `app.js` with the following code.

```
function App(){
  const name = "John";
  return (
    <div>
      <p>Name is : {name}</p>
    </div>
  )
}
export default App;
```

COPY

Printing a Boolean value

Replace the `app.js` with the following code.

```
function App(){
  const x = true;
  return (
    <div>
      <p>{x.toString()}</p>
    </div>
  )
}
export default App;
```

COPY

Printing an Array

COPY

```
function App(){
  const course = [ "python", "java", "php", "javascript" ];
  return (
    <div>
      <p>{course[1]}</p>
    </div>
  )
}
```

export default App;

Conditional printing with if else

In this example, we will try to bring an if condition within the code
Replace **App.js** with the following code

COPY

```
import './App.css';
import image from './images/mohanlal.jpg'
function App() {
  let greeting = "Hello World";
  let showGreeting = true;
  if(showGreeting){
    var greetingCode = <h1 className="greetingStyle">{greeting}</h1>
  }else{
    var greetingCode = null
  }
}
```

The above code will print the "Hello World" if the variable is set to true.

Activity

Try changing the value of the variable to false and see the output

← PREV

NEXT →

← PREV

NEXT →

Printing from an array of strings

Replace **App.js** with the following code

COPY

```
import './App.css';
import image from './images/mohanlal.jpg'
function App() {
  let greetings = ["Good Morning", "Good Afternoon","Good Evening", "Good Night", "Good Day"];

  return (
    <div>
      <img src={image}/>
      {greetings[2]}
    </div>
  )
}
```

The above code will print the third element of the array.

Printing the whole array using map() method

Replace **App.js** with the following code

COPY

```
import './App.css';
import image from './images/mohanlal.jpg'
function App() {
  let greetings = ["Good Morning", "Good Afternoon","Good Evening", "Good Night", "Good Day"];

  return (
    <div>
      <img src={image}/>
      {greetings.map((item)=>{return <p>{item}</p>})}
    </div>
  )
}
```

The above code will print the entire contents of the array

Printing the whole array using for loop

Replace **App.js** with the following code

COPY

```
import './App.css';
import image from './images/mohanlal.jpg'
function App() {
  const fruits = ["apple", "banana", "cherry", "mango"];
  const fruitList = [ ];
  for( let i=0; i < fruits.length; i++) {
    fruitList.push(<li key ={i}> {fruits[i]} </li> );
  }
  return (
    <div>
      <img src={image}/>
      {fruitList}
    </div>
  )
}
```

This will print the array elements as a list.

✔ Activity

Use the array to create a picture gallery

Printing from an array of objects

Replace **Apps.js** with the following code

```
import './App.css';
import image from './images/mohanlal.jpg'
function App() {
  let greetings = [{name:"Vijay", location:"Kochi"}, {name:"Rajesh", location:"Delhi"}];

  return (
    <div>
      <img src={image}/>
      {greetings[1].name}
    </div>
  )
}
```

COPY

The above code will print Rajesh as the output

Printing an whole array of objects

Replace **App.js** with the following code

```
import './App.css';
import image from './images/mohanlal.jpg'
function App() {
  let students = [{name:"Vijay", location:"Kochi"}, {name:"Rajesh", location:"Delhi"}];

  return (
    <div>
      <img src={image}/>
      {students.map((item)=>{return <p>{item.name} is from {item.location}</p>})}
    </div>
  )
}
```

COPY

✔ Activity

Show the above students array as a table with name as the first column, location as the second column

← PREV

NEXT →

← PREV

NEXT →

Setting html attribute value from variable

App.js

```
import React from 'react';

function App(){
  var user = 'John';
  return (
    <div>
      <input type='text' value={user} />
    </div>
  )}
  . . . . .
```

COPY

Setting html attribute value from array

App.js

```
import React from 'react';

function App(){
  var user = ["John" , "William", "David"];
  return (
    <div>
      <input type="text" value={user[1]} />
    </div>
  )}
  . . . . .
```

COPY

Setting attribute value from object

App.js

```
import React from 'react';

function App(){
  var user = { name : "Arun" , age : 20 }
  return (
    <div>
      <input type="text" value={user.name} />
    </div>
  )}
  . . . . .
```

COPY

Events

Responding to various events

Click

Write the following code in VS Code

COPY

```
function App() {
  return (
    <div>
      <button onClick={()=>{console.log("welcome")}}>Click Me</button>
    </div>
  );
}
export default App;
```

The above code will print "welcome" in the console when the button is clicked.

Other Events

To change click to other events, change onClick to corresponding event functions

Activity

Change the above code for double click

Change the above code for hover

Calling Functions On Events

In React, you can call functions when certain events occur by defining event handlers. Event handlers are functions that are executed when a specific event is triggered, such as clicking a button or hovering over an element.

App.js

COPY

```
function handleClick() {
  console.log('Button was clicked!');
}

function App() {
  return (
    <button onClick={handleClick}>
      Click me
    </button>
  );
}
```

PREV

NEXT

← **PREV**

NEXT →

State

In React, State is an object that stores the dynamic data that change over time and affect the behaviour of a component

A component can use state to keep track of its own data and update its UI accordingly. The Functional components use useState() hook to store its state. To use the hook import it from react.

```
import React, { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
  }
}
```

COPY

Here, the "count" is the 'current state' and 'setCount' is the function to update the state. 0 is the initial value of the state. When the button is clicked, setCount(count + 1) is called, which updates the count state by incrementing it by 1. This causes React to re-render the component and update the UI with the new value of count.

Outputting the value entered by the user in input field

```
import React, { useState } from 'react';

function App() {
  const [name, setName] = useState('');

  function handleNameChange(event) {
    setName(event.target.value);
  }
}
```

COPY

In the above code , we have a form element (an input) whose value is bound to the state variable name. We also have an onChange event handler called handleNameChange which updates the value of name whenever the user types something into the input. Finally, we render the value of name inside a paragraph tag to display a greeting to the user.

Form

COPY

```
import React, { useState } from 'react';
import './App.css';

function App() {
  const [name, setName] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Hello, ${name}!`);
  }
```

In the above code, we use the `useState()` hook to manage the form data as state. The `handleSubmit` function is called when the form is submitted and can be used to send the form data to the server.

Splitting a page into multiple components

In react, a common way to split a page into multiple components is by using a component-based architecture. This allows us to break our UI down into smaller, reusable parts that can be composed together to create more complex interfaces.

COPY

```
import React from 'react';
import Header from './Header';
import MainContent from './MainContent';
import Footer from './Footer';

function App() {
  return (
    <div>
      <Header />
      <MainContent />
      <Footer />
    </div>
  );
}
```

Rendering a component in another component

To display a component within another component, you can simply render the child component as a JSX element within the parent component's render method. Import the component which needs to be rendered in the parent component.

App.js

COPY

```
import React from 'react';
import Header from './Header';

function App(){
  return (
    <div>
      <Header/>
    </div>
  )
}
```

← PREV

NEXT →

← PREV

NEXT →

Props

In react, props (short for "properties") are a way to pass data from one component to another.

A parent component can pass data to a child component through props. The child component can then access the data by using the props object. The data in props is read-only meaning that a child component cannot modify the data passed to it through props.

Passing data from parent component to child component

Here is an example of how to use props in components.

// Parent Component

COPY

```
import React from 'react';
import Child from './child';

function App() {
  const name = "Alice";
  const age = 25;
  return (
    <Child name={name} age={age} />
  );
}
```

//Child component - create a component named child.js and add the following code in it.

COPY

```
import React from 'react';

function Child(props) {
  return (
    <div>
      <p> Name : {props.name} </p>
      <p>Age : {props.age} </p>
    </div>
  );
}
```

In the above example, the parent component passes the name and age variables to the child component through props. The child component then accesses these values through the props object. Props can also be used to pass functions as parameters to child components. This allows child components to interact with the parent component in various ways.

Displaying child nodes of a component

To display child nodes of a component using props in reactJS, you can pass the child nodes as a prop to the parent component and render them using the props.children property.

// Parent Component

COPY

```
import React from 'react';
import Child from './child';

function App() {
  const name = "Alice";
  const age = 25;
  return (
    <Child name={name} age={age} >
      <p> Welcome to mashup stack </p>
    </Child>
  );
}
```

//Child component

COPY

```
import React from 'react';

function Child(props) {
  return (
    <div>
      <p> Name : {props.name} </p>
      <p>Age : {props.age} </p>
      <p>{props.children}</p>
    </div>
  );
}
```

Passing function reference through props

In React, you can pass a function reference as a prop from a parent component to a child component, allowing the child component to call the function defined in the parent component.

// Parent Component

COPY

```
import React, {useState} from 'react';
import Child from './child';

function App() {
  const [count, setCount] = useState(0);
  function incrementCount(){
    setCount(count + 1);
  }
  return (
    <div>
      <Child increment={incrementCount}>
        <p> Count is {count} </p>
      </Child>
    </div>
  );
}
```

//Child component

Child.js

COPY

```
import React from 'react';

function Child(props){
  return(
    <button onClick={props.increment}> Increment count </button>
  )
}
export default Child;
```

Calling a function of parent component from child component

To call a function of a parent component from a child component in react, you can pass the function down to the child component as a prop. Replace the app.js and child.js with the following code.

App.js

COPY

```
import React , { useState } from 'react';
import Child from './child';

function App() {
  const [count, setCount ] = useState(0);
  function incrementCount () {
    setCount(count+1);
  }
}
```

Child.js

COPY

```
import React from 'react';

function Child(props) {
  function handleClick() {
    props.increment()
  }
  return (
    <button onClick = {handleClick} >Increment Count </button>
  );
}
```

In the above example, the incrementCount function is defined in the parent component and passed down to the child component as a prop called increment. The child component then uses this prop to call the incrementCount function when a button is clicked.

When a button is clicked , the handleClick function in the child component is called, which in turn calls the increment prop passed down from the parent component. This triggers the incrementCount function in the parent component which updates the count state and causes the component to rerender with the updated count value.

Passing data from child to parent component

To pass data from a child component to the app component in a react application, you can use a callback function that is passed from the app component to the child component as a prop. Here is an example of how to do this:

In the app component, define a function that will receive data from the child component. This function will be passed as a prop to the child component.

App.js

COPY

```
import React, { useState } from 'react';
import Child from './child';

function App() {
  const [ dataFromChild, setDataFromChild] = useState(null);

  const handleDataFromChild = (data) => {
    setDataFromChild(data);
  }
}
```

In the child component, define a function that will be called when the data needs to be passed to the app component. This function will use the prop passed from the app component to call the callback function with the data as an argument.

Child.js

COPY

```
import React, { useState } from 'react';

function Child(props) {
  const [data, setData] = useState(null);
  const handleClick = () => {
    const newData = " mashup stack";
    setData(newData);
    props.onDataFromChild(newData);
  }
}
```

Here, when the button is clicked in the child component, the handleClick function is called, which sets the data state and calls the onDataFromChild prop passed from the app component with the newData value as an argument. This triggers the handleDataFromChild function in the app component, which sets the dataFromChild state to the received value, causing it to be displayed in the p element.

← PREV

NEXT →

← **PREV**

NEXT →

useEffect() hook

The `useEffect()` hook is used to manage side effects in functional components in react.

It allows you to perform some actions after the component has rendered or rerendered.

When a component is rendered for the first time, all the `useEffect` hooks in the component are executed.

When a component is rerendered, the `useEffects` hooks are executed again.

However, before executing the `useEffect` hooks, react first compares the previous values of the dependencies specified in the `useEffect` hook with the new values of those dependencies.

If there is no change in the values of the dependencies, then the `useEffect` function is not executed again.

If the values of the dependencies have changed, then the `useEffect` function is executed again.

This is useful when you want to perform some action only when certain dependencies change. For example, you might want to fetch new data from an API only when a certain prop changes. In this case, you would include that prop in the dependency array of the `useEffect` hook.

Do something when the component is first rendered

Replace the `app.js` with the following code.

```
import { useState, useEffect } from 'react';

function App(){
  const [count, setCount] = useState(0);
  useEffect( () =>{
    console.log("component has rendered");
  }, [ ]);

  const handleClick = () => {
    // ...
  }
}
```

COPY

Do something when the component is re-rendered

```
import { useState, useEffect } from 'react';

function App(){
  const [count, setCount] = useState(0);
  useEffect( () =>{
    console.log("component has rendered");
  });

  const handleClick = () => {
    // ...
  }
}
```

COPY

In the above code, we have removed the dependency array so the `useEffect` will be executed again after the component has rerendered.

Do something with `useEffect` when the state value changes


```
import React, { useState, useEffect } from 'react';

function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log(`Count changed to ${count}`);
    // Do something here after the count state value changes
  }, [count]);
```

In the above example, the count state value is initialised to 0, and every time it changes, the useEffect hook is called with a callback function that logs a message to the console. The second argument to useEffect is an array of dependencies, which includes only the count variable. This tells useEffect to only execute the callback function when the count variable changes.

When the "IncrementCount" button is clicked, the count state value is updated using the setCount function, which triggers the useEffect hook to execute its callback function with the updated count value.

[< PREV](#)[NEXT >](#)

← PREV

NEXT →

Routing

Routing in react refers to the process of dynamically changing the content of a web page based on the URL.

React Router DOM is a third-party library. It is not a part of the official React library but is widely used in the React ecosystem for handling routing in single-page applications.

To use React Router, you need to install it by running the following command in your terminal.

```
npm install react-router-dom
```

COPY

Mapping URLs to components

What we will do here is that we will create two urls of which one will show the app component to the user and the other url will show the About us component to the user. Create a folder named components inside the src folder.

Create a About us component

components/Aboutus.js

```
import React from 'react';

function Aboutus() {
  return (
    <div>
      <h1>About Us</h1>
      <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit
    </div>
  );
}
```

COPY

App.js

```
import React from 'react';

function App() {
  return(
    <div>
      <p>Home component </p>
    </div>
  );
}
```

COPY

Create a router.js file in the component directory

This file will be used to map the urls to the components. Each mapped url is called a route.

router.js

COPY

```
import { createBrowserRouter } from "react-router-dom";
import Aboutus from "../components/Aboutus";
import App from "../App";

const router = createBrowserRouter([
  { path: '/', element: <App/> },
  { path: 'aboutus', element: <Aboutus/> },
]);
```

Here, the routes are written as an object. The url is in the path property of the object and the component is in the element property of the object. Next we will specify where the components are to be shown. We will use routerProvider from react-router-dom for this purpose. Import the routerProvider from react-router-dom

Open your index.js file and replace the app with routerprovider

index.js

COPY

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import reportWebVitals from './reportWebVitals';
import { RouterProvider } from 'react-router-dom';
import router from '../components/router';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
```

Creating links to routes

We can't always ask the user to type in the urls to see the page. We need to provide them with links. When we use tag the page gets reloaded which defeats the purpose of the SPA. We can use 'Link to' which will prevent the default action of the 'a' tag.

Replace the Aboutus component with the following code.

Aboutus.js

COPY

```
import React from 'react';
import { Link } from "react-router-dom";

function Aboutus() {
  return (
    <div>
      <h1>About Us</h1>
      <p>
        Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do
        eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad
        minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip
        ex ea commodo consequat. Duis aute irure dolor in reprehenderit in
        voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur
        sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt
        mollit anim id est laborum.
      </p>
    </div>
  );
}
```

Creating a Navbar component

Lets now create a navbar component. This component will contain links to all other components. Create a new file named Navbar.js inside the components folder and add the following code in it.

Navbar.js

COPY

```
import { NavLink } from "react-router-dom";

function Navbar() {
  return <nav className="navbar navbar-expand-sm navbar-dark bg-dark">
    <div className="navbar-brand">
      <h4>My app</h4>
    </div>
    <button
      className="navbar-toggler"
      type="button"
      onClick={toggleNavbar}
      aria-label="Toggle navigation">
      <span className="navbar-toggler-icon"></span>
    </button>
    <div className="navbar-collapse">
      <ul>
        <li><NavLink to="/">Home</NavLink></li>
        <li><NavLink to="/about">About</NavLink></li>
        <li><NavLink to="/contact">Contact</NavLink></li>
      </ul>
    </div>
  </nav>
```

App.js

```
import React , { useState } from 'react';
import Navbar from './components/Navbar';

function App() {
  return(
    <div>
      <Navbar/>
      <p>Home component </p>
    </div>
  )
}
```

COPY

← PREV

NEXT →

← PREV

NEXT →

Styling links based on current url

Here, we are going to style the links based on their active state.

Navbar.js

COPY

```
import { NavLink } from "react-router-dom";

function Navbar() {
  return <nav className="navbar navbar-expand-sm navbar-dark bg-dark">
    <div className="navbar-brand">
      <h4>Todo List</h4>
    </div>
    <button className="navbar-toggler" type="button" data-toggle="collapse"
      data-target="#navbarNav" aria-controls="navbarNav"aria-expanded="false"
      ...>
```

Programmatic navigation

Consider the scenario where the user needs to be redirected to a page automatically based on some condition. For example, consider authentication. When the user submits the login form they should be redirected to the home page automatically. Clicking any link to go to the home page will affect the user experience. To facilitate programmatic navigation, we can use the useNavigate() hook from react-router-dom.

Aboutus.js

COPY

```
import { Link, useNavigate } from "react-router-dom";
import Navbar from "../Navbar";

function Aboutus() {
  const navigate = useNavigate();
  function goToHomePage(){
    navigate('/')
  }
  return <div>
```

Parameterised Route

In React, parameterised routes allow you to create dynamic routes that can accept different parameters or values, such as userIDs, product IDs, or any other data that needs to be passed between components. Create a component named greetings.js inside the components directory.

Greetings.js

COPY

```
import { useParams } from "react-router-dom";

function Greetings() {
  const params = useParams();
  return <h1>Hello {params.name}</h1>
}
export default Greetings;
```

Here, we have used the useParams() hooks to get the parameter from the route.

router.js

Add this path in the router file.

```
{ path: 'greeting/:name', element: <Greetings/>}
```

COPY

Here, the greeting is fixed and :name is the parameter. The parameter can be any string. Parameters are always preceded by (:)

Open your browser and enter the following url

http://localhost:3000/greeting/mashupstack

← PREV

NEXT →

← PREV

NEXT →

Simple CRUD

For doing simple crud operations create a file named Crud.js in the components folder.

Now let's do a list of the data in the state. CRUD stands for Create, Read, Update, and Delete.

List

```
import React, { useState } from "react";
import Navbar from './Navbar';

function Crud() {
  var [ items, setItems ] = useState ([
    { id : 1, name : "John" },
    { id : 2, name : "David" },
    { id : 3, name : "William" }
  ]);
```

Here we have listed the values from the state using the map method and shown the result in the table.

Create

Crud.js

```
import React, { useState } from "react";
import Navbar from './Navbar';

function Crud() {
  var [ items, setItems ] = useState ([
    { id : 1, name : "John" },
    { id : 2, name : "David" },
    { id : 3, name : "William" }
  ]);
```

Here, we have used the useState hook to store the values. Input tag is used to collect the value from the user. The inputted value will be handled by the handleInputChange function on the onChange event. Once the form get submitted the inputted value will be stored in the state.

Delete

```
import React, { useState } from "react";
import Navbar from './Navbar';

function Crud() {
  var [ items, setItems ] = useState ([
    { id : 1, name : "John" },
    { id : 2, name : "David" },
    { id : 3, name : "William" }
  ]);
```

Edit

```
import React, { useState } from "react";
import Navbar from './Navbar';

function Crud() {
  var [ items, setItems ] = useState ([
    { id : 1, name : "John" },
    { id : 2, name : "David" },
    { id : 3, name : "William" }
  ]);
```

COPY

Search

```
import React, { useState } from "react";
import Navbar from './Navbar';

function Crud() {
  var [ items, setItems ] = useState ([
    { id : 1, name : "John" },
    { id : 2, name : "David" },
    { id : 3, name : "William" }
  ]);
```

COPY

← PREV

NEXT →

← **PREV**

NEXT →

Axios

Axios is a popular Javascript library used for making HTTP requests from web browsers and Node.js applications.

It provides a simple and easy-to-use API that allows developers to send asynchronous requests to servers and receive responses in a variety of formats including JSON, XML, and others. Axios is promise based and thus we can take advantage of async and await for more readable asynchronous code.

To install axios run the following command in the terminal.

```
npm install axios
```

COPY

Next we need to import it in the component where we are sending a request.

import axios from "axios"

Axios uses functions, in the name of Http methods to send the request.

```
axios.get();
```

COPY

The Url is passed as the first parameter

```
axios.get('https://demo-blog.mashupstack.com/api/posts');
```

COPY

Body is passed as the second parameter. Note that the body is not present for the get and delete method.

```
let value = {
  title: "Lion",
  content: "The lion is often called the king of the forest because of its strength, power and majestic appearance.",
};

axios.post('https://demo-blog.mashupstack.com/api/posts', value);
```

COPY

Headers are passed inside the options object as the third parameter for the **post, put, patch** requests and second parameter for get and delete requests.

```
let value = {
  title: "Lion",
  content: "The lion is often called the king of the forest because of its strength, power and majestic appearance.",
};

let options = {
  headers: { accept: 'application/json' },
};

axios.post('https://demo-blog.mashupstack.com/api/posts', value, options);
```

COPY

These functions return a promise, so we will use the then() method to handle the success response and catch() method to handle the error response. For example,

COPY

```

    axios.get('https://demo-blog.mashupstack.com/api/posts')
      .then(response => {
        setState(response.data); // Updating state using the `setState` updater function returned by the useState hook.
      })
      .catch(error => {
        console.log(error);
      });
  
```

Lets see some of the examples.

Get request

COPY

```

    var options = {
      headers: {'Accept': 'application/json'}
    }
    axios.get('url-of-the-api-endpoint', options).then(response=>{
      // statements to execute on success response from api endpoint
      console.log(response.data)
    }).catch(error=>{
      // statements to execute on error response from api endpoint
      console.log(error)
    })
  
```

Get request with query params

COPY

```

    var options = {
      headers: {'Accept': 'application/json'},
      params: {
        'key': 'value'
      }
    }
    axios.get('url-of-the-api-endpoint', options).then(response=>{
      // statements to execute on success response from api endpoint
      console.log(response.data)
    })
  
```

Post request with body as formdata

COPY

```

    var data = new FormData();
    data.append('email', 'demo@mashupstack.com');
    data.append('password', '123456');
    var options = {
      headers: {'Accept': 'application/json'},
    }
    axios.post('url-of-the-api-endpoint', data, options).then(response=>{
      // statements to execute on success response from api endpoint
      console.log(response.data)
    })
  
```

Post request with body as Json

COPY

```
var data = {
  email: 'demo@mashupstack.com',
  password: '123456'
};
var options = {
  headers: {'Accept': 'application/json'},
}
axios.post('url-of-the-api-endpoint', data, options).then(response=>{
  // statements to execute on success response from api endpoint
})
```

Post request with body as url encoded

COPY

```
const axios = require('axios'); // Make sure you require axios if using Node.js

const data = new URLSearchParams({
  email: 'demo@mashupstack.com',
  password: '123456'
});

const options = {
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded'
  }
}
```

Data Formatting: URLSearchParams converts the object into a URL-encoded string

← PREV

NEXT →

COPY

```
import { NavLink } from "react-router-dom";

function Navbar() {
  return <nav className="navbar navbar-expand-sm navbar-dark bg-dark">
    <div className="navbar-brand">
      <h4>Todo List</h4>
    </div>
    <button className="navbar-toggler" type="button" data-toggle="collapse"
data-target="#navbarNav" aria-controls="navbarNav"aria-expanded="false"
data-bs-toggle="collapse" data-bs-target="#navbarNav" aria-expanded="false"
data-bs-keyboard="true" data-bs-parent="#navbarNav">
```

Create post

Next we will add a component to create a post. We need a form to get the title and content from the user.We will use axios to send them to the api to create a post. We will redirect to listing page when the post is created.

src/components/blog/CreatePost.js

COPY

```
import axios from "axios";
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import Navbar from "../Navbar";

function CreatePost() {
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  var navigate = useNavigate()
  // ... (rest of the component code) ...
}
```

Add the following path to the router file

Router.js

COPY

```
{ path : 'blog/posts/create' , element : <CreatePost/> }
```

View Post

Next is the component to view the post. We will get the post id from the route and useEffect hook to get the post details using axios and display it.

src/components/blog/Viewpost.js

COPY

```
import axios from "axios";
import { useEffect, useState } from "react"
import { useParams } from "react-router-dom"
import Navbar from "../Navbar";

function ViewPost() {
  var {postId} = useParams()
  var [post,setPost] = useState({title:'',content:''})
  useEffect(()=>{
    // ... (rest of the component code) ...
  }, [])
}
```

Add the view page route to the router file

COPY

```
{ path: 'blog/posts/:postId', element: <ViewPost/>},
```

Edit post

Next is the component to edit the post. Here we need to fetch the details from the api and populate the form and on submitting the form the new details should be sent to the api to update the post.

```
import axios from "axios";
import { useEffect, useState } from "react";
import { useParams, useNavigate } from "react-router-dom";
import Navbar from "../Navbar";

function EditPost() {
  const {postId} = useParams();
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  // ... (rest of the component code)
}
```

COPY

Add the edit route path to the router file

```
{ path : '/blog/posts/:postId/edit', element: <EditPost/>}
```

COPY

Search post

```
import axios from "axios";
import { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import Navbar from "../Navbar";
import PostListItem from "../PostListItem";

function ListPosts() {
  const [allPosts, setAllPosts] = useState([]); // Store all the fetched posts from the API
  const [filteredPosts, setFilteredPosts] = useState([]); // Store the filtered posts based on search term
  // ... (rest of the component code)
}
```

COPY

Here, we have a search field and two buttons for search and reset. When the search button is clicked the handleSearch function is invoked. It filters the data and displays it. When the reset button is clicked the handleResetSearch gets invoked. This function calls the fetchPosts function which lists all the posts in the database.

← PREV

NEXT →

← **PREV**

NEXT →

Authentication

Authentication is the process of identifying the user accessing the site and restricting the content to certain users.

Authentication is an important aspect of security and is used to protect sensitive information and resources from unauthorized access. We can't simply give access to a user. They need to provide username and password. These are then send to the login api. The api verifies whether the username and password is correct and if they are correct it returns a token. Therefore this token needs to be send with every request to the server that is available only for logged in users.

In this chapter we will do the following things:

Registering a user

- Create a login component to show the login form
- Create an auth slice to store the user details and save the token to it on successful login.
- Modify navbar to login and logout links
- Modify blogs to show it only for logged in users.
- Attach token to api calls
- Save token to local storage
- Retrieve token from local storage

Registering a user

This party is very easy.You just need to collect the user details from the form and send it to the register api.On successful registration the user is redirected to the home page. Ideally it should be redirected to the login page but we have not yet created it.

components/auth/register.js

COPY

```
import axios from "axios";
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import Navbar from "../Navbar";

function Register() {
  var [name, setName] = useState('');
  var [email, setEmail] = useState('');
  var [password, setPassword] = useState('');
  // ...
}
```

Next step is to add a route for the component and add it to the navbar

router.js

COPY

```
{ path: 'register', element:<Register/>}
```

src/components/navbar.js

Navbar.js

COPY

```
<li className="nav-item">
  <NavLink
    to={"/register"}
    className={
      'nav-link '+
      (status => status.isActive ? 'active' : '')
    }
  >
    Register
  </NavLink>
</li>
```

← PREV

NEXT →

← PREV

NEXT →

Creating the login form

Creating a login form with provision to show error messages.

components/auth/Login.js

COPY

```
import { useState } from "react";
import Navbar from "../Navbar";

function Login() {
  var [email, setEmail] = useState('');
  var [password, setPassword] = useState('');
  var [errorMessage, setErrorMessage] = useState('');
  function attemptLogin() {
```

When the form is submitted the login credentials should be sent to the api using axios. Modify the attemptLogin function as follows.

COPY

```
function attemptLogin() {
  axios.post('https://demo-blog.mashupstack.com/api/login', {
    email:email,
    password:password
  }).then(response=>{
    setErrorMessage('')
    console.log(response.data.token)
  }).catch(error=>{
    if(error.response.data.errors){
```

Here, when attemptLogin is called, it will send the credentials to the login api and show an error message if the requests fail. If the request is successful the error msg is emptied. Next we will save the token to the application store.

← PREV

NEXT →

[< PREV](#)[NEXT >](#)

Redux

Redux is a predictable state management library for JavaScript applications, commonly used with React but also compatible with other libraries and frameworks. It helps manage the state of an application in a more organized and structured manner, making it easier to reason about the data flow and changes within the application.

The core idea behind Redux is to maintain the entire application state in a single immutable state tree, also known as the "store".

The state tree represents the state of the entire application and is read-only. To make any changes to the state, you need to dispatch an action. Actions are plain JavaScript objects that describe what happened in the application, such as a user clicking a button, typing in a form, or fetching data from a server.

When an action is dispatched, it flows through a pure function called the "reducer". The reducer takes the current state and the action as input and returns a new state that reflects the changes caused by the action. Reducers are pure functions, meaning they do not modify the original state but create a new state based on the input.

The Redux store holds the state returned by the reducer and notifies any subscribers when the state changes. Components that need access to the state can subscribe to the store and receive updates whenever the state changes.

Key concepts in Redux:

- 1. Store:** The single source of truth that holds the entire application state. It is read-only and can only be changed by dispatching actions.
- 2. Action:** Plain JavaScript objects that describe what happened in the application. Actions must have a `type` property that indicates the type of action being performed.
- 3. Reducer:** A pure function that takes the current state and an action as input and returns a new state based on the action type. Reducers should not have side effects.
- 4. Dispatch:** The process of sending an action to the Redux store to update the state. It is done using the `store.dispatch()` method.

Redux simplifies state management by enforcing a unidirectional data flow, making it easier to debug and test applications. While it may introduce some boilerplate code, it offers clear patterns for managing complex state interactions in larger applications. It has become a popular choice for state management in many JavaScript applications due to its predictability and ease of use.

[< PREV](#)[NEXT >](#)

← **PREV**

NEXT →

Creating Auth Slice

To save the token in the store, you need to create a slice. Let's create an authSlice.js to store the details of the logged in user. The initial value of user will be null as no user has logged in yet.

src/store/authSlice.js

COPY

```
import { createSlice } from "@reduxjs/toolkit";

export const authSlice = createSlice({
  name: 'auth',
  initialState: {
    user: null
  },
  reducers: {
  }
});
```

For our purpose, we need two actions

Action to save user details

Action to remove user details

Let's add the action to add the user details and remove the user details to the authSlice.

src/store/authSlice.js

COPY

```
import { createSlice } from "@reduxjs/toolkit";

export const authSlice = createSlice({
  name: 'auth',
  initialState: {
    user: null
  },
  reducers: {
    setUser: (state, action) => {
    }
  }
});
```

Next we need to add this slice to the store. Open store.js and add the following.

src/store/store.js

COPY

```
import { configureStore } from "@reduxjs/toolkit";
import authReducer from './authSlice';

var store = configureStore({
  reducer: {
    auth: authReducer
  }
});
export default store;
```

Next we need to modify the attemptLogin function in login component to save user details on login

src/components/auth/Login.js

COPY

```
import axios from "axios";
import { useState } from "react";
import { useDispatch } from "react-redux";
import { setUser } from "../../../store/authSlice";
import Navbar from "../../../Navbar";
import { useNavigate } from "react-router-dom";

function Login() {
  var [email, setEmail] = useState('');
  // ... (rest of the code) ...
}
```

← PREV

NEXT →

← **PREV**

NEXT →

Login and logout links in navbar

Login link should be shown only if the user is not logged in. In other cases the log out link should be shown. To find whether the user is logged in or not, get the user from auth slice.

src/components/Navbar.js

COPY

```
var user = useSelector(store=>store.auth.user);
```

If the user in the slice is empty, the user is not logged in. If it is not empty, the user is logged in.

COPY

```
{user?
  <li className="nav-item">
    <span className="nav-link">Logout</span>
  </li>:
  <li className="nav-item">
    <NavLink to={"/login"}
      className={
        'nav-link '+
        (status => status.isActive ? 'active' : '')
      }
    >
```

Clicking the login link should redirect the user to the login form. We added a navlink above.Now we need to map the /login url to the login form component in router.js.

COPY

```
{ path: 'login', element:<Login/>}
```

Clicking the Logout link in Navbar should do the following

- Call the logout api using axios
- Remove the user details from store
- Redirect user to the login page

We will create a function to do these things and bind the function to the click of logout link in Navbar/

Navbar.js

COPY

```
const dispatch = useDispatch();
const navigate = useNavigate();
function logout(){
  if(user){
    axios.post('https://demo-blog.mashupstack.com/api/logout',{},{
      headers:{'Authorization':"Bearer "+ user.token}
    });
    dispatch(removeUser());
    navigate('/login');
```

Navbar.js

COPY

```
<li className="nav-item">
  <span className="nav-link" onClick={logout}>Logout</span>
</li>
```

You might have noticed this line headers:{'Authorization':"Bearer"+ user.token} in the above code. This is to send the auth token with the request. We will learn more about it later.

The navbar component now looks like this.

src/components/Navbar.js

COPY

```
import axios from "axios";
import { useDispatch, useSelector } from "react-redux";
import { NavLink, useNavigate } from "react-router-dom";
import { removeUser } from "../store/authSlice";

function Navbar() {
  var user = useSelector(store=>store.auth.user);
  const dispatch = useDispatch ();
  const navigate = useNavigate();
  // ...
}
```

Modify your index.js like this

COPY

```
<Provider store={store}>
  <AutoLogin>
    <RouterProvider router={router}/>
  </AutoLogin>
</Provider>
```

← PREV

NEXT →

← **PREV**

NEXT →

Restricting Access to a component to logged in users

There will be some pages that should only be shown to logged in users and some pages that should only be shown to non logged in users. Here we will learn how to restrict access to a component.

As an example we will restrict blog pages only to logged in users. We will also restrict login page access to non logged in users. React does not have an out of solution for it. So We will have to use either a third party package or write our own solution. Here we will write our own solution.

We will use a function that accepts a function as its parameter.

```
export const checkAuth = (Component) =>{  
  
}
```

This function should check if the user is logged in and return the component if logged in otherwise redirect to login. For this we have to follow the react hooks useEffect, useSelector, useNavigate. But hooks can only be used inside a component and checkAuth is not one. So inside the checkAuth function we will create another component wrapper. The hooks will be called inside the wrapper component.It will then render the component passed as argument of checkAuth. The checkAuth function will return the function component.

COPY

```
export const checkAuth = (Component) =>{  
  function Wrapper(props){  
    var user = useSelector(store=>store.auth.user);  
    var navigate = useNavigate();  
    useEffect(()=>{  
      if(!user){  
        navigate('/login');  
      }  
    }, [user]);  
    return <Component {...props} />;  
  }  
  return Wrapper;  
}
```

Did you notice this line . Here props is the property of the wrapper component. We are merging this with the properties of components. Inside the useeffect hook we will check if the user is logged in and redirect to the login page if the user is empty.

Now we can pass the components that need to be restricted to the function. Since this function is needed for multiple components, we will write it in a separate file.

src/components/auth/checkAuth.js

COPY

```
import { useEffect } from "react";  
import { useSelector } from "react-redux";  
import { useNavigate } from "react-router-dom"  
  
export const checkAuth = (Component) =>{  
  function Wrapper(props){  
    var user = useSelector(store=>store.auth.user);  
    var navigate = useNavigate();  
    useEffect(()=>{  
      if(!user){  
        navigate('/login');  
      }  
    }, [user]);  
    return <Component {...props} />;  
  }  
  return Wrapper;  
}
```

To restrict the post listing page of a blog, open the ListPost component and modify the export statement. Note we need to import the checkAuth function.

src/component/blog/ListPost.js

COPY

```
import checkAuth from "../auth/checkAuth"
.
.
.
export default checkAuth(ListPost);
```

Now restrict the EditPost, CreatePost and ViewPost components just like above.

Now let's create another function similar to the one above to check for non logged in users. Then use it to restrict the login page.

src/components/auth/checkguest.js

COPY

```
import { useEffect } from "react";
import { useSelector } from "react-redux";
import { useNavigate } from "react-router-dom"

export const checkGuest = (Component) =>{
  function Wrapper(props){
    var user = useSelector(store=>store.auth.user);
    var navigate = useNavigate();
    useEffect(()=>{
      // ...
    })
  }
}
```

Now open the Login.js file and modify the export statement.

src/components/auth/Login.js

COPY

```
import axios from "axios";
import { useState } from "react";
import { useDispatch } from "react-redux";
import { setUser } from "../../../store/authSlice";
import Navbar from "../Navbar";
import checkGuest from "../checkGuest";

function Login() {
  var [email, setEmail] = useState('');
  // ...
}
```

← PREV

NEXT →

← PREV

NEXT →

Attach token to api calls

As we saw in the beginning of the chapter, the token received from the login api should be attached to the request that needs authentication. Earlier we saw the logout api the token is added to the header. Different apis have different ways to attach token.

It will be specified in the api documentation. In our api it is passed as a bearer token. Bearer token uses the Authorization token. The token string should be present in the header's value. The token string will be preceded by the word Bearer separated by space.

When we use it in code it will look like this

```
axios.post('https://demo-blog.mashupstack.com/api/logout',{},{ headers:{'Authorization':"Bearer "+ user.token} });
```

The token is retrieved from store.

Save token to local storage

Authentication is almost over. There is a problem with this. The token is saved in the store. The store gets reset if we reload the page. This is not relevant in normal cases as we are building a single page application and we don't usually need to reload pages. But what if user accidentally closes the tab.

When he reopens it he will have to login again. To prevent this what we do is that, we will save the user details to the local storage also. The actions of the auth slice will now look like this.

```
setUser: (state,action)=>{ state.user = action.payload; window.localStorage.setItem('user',JSON.stringify(action.payload)) }, removeUser: (state)=>{ state.user = null; window.localStorage.removeItem('user') },
```

src/store/authSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

export const authSlice = createSlice({
  name: 'auth',
  initialState:{
    user: null
  },
  reducers:{
    setUser: (state,action)=>{
      state.user = action.payload;
      window.localStorage.setItem('user',JSON.stringify(action.payload))
    },
    removeUser: (state)=>{
      state.user = null;
      window.localStorage.removeItem('user')
    }
  }
})
```

COPY

← PREV

NEXT →

← PREV

NEXT →

Set saved user on app start

When we saved the user details to local storage. Now we will get this user on app start and set the auth slice user value.

First we need to add another action to the authSlice.

```
setUserFromLocalStorage: (state)=>{
  var user = window.localStorage.getItem('user');
  if(user){
    user = JSON.parse(user);
    state.user = user;
  }else{
    state.user = null;
  }
}
```

COPY

The authSlice will now look like this

```
import { createSlice } from "@reduxjs/toolkit";

export const authSlice = createSlice({
  name: 'auth',
  initialState:{
    user: null
  },
  reducers:{
    setUser: (state,action)=>{
      state.user = action.payload
    }
  }
})
```

COPY

This will get the value from localStorage, parse it as an object and save it to the state. Now we need to dispatch it from the root component. We can use the useEffect hook in the root component to do this. But our root component is RouterProvider which is defined in the react-router-dom package. We cannot edit it. So we will create another component like this.

src/components/auth/AutoLogin.js

```
import { useEffect } from "react";
import { useDispatch } from "react-redux";
import { setUserFromLocalStorage } from "../../store/authSlice";

function AutoLogin(props){
  const dispatch = useDispatch();
  useEffect(()=>{dispatch(setUserFromLocalStorage())}, [])
  return props.children
}
```

COPY

This component dispatches the setUserFromLocalStorage and displays its child component.

The index.js should look like this.

```
<Provider store={store}>
  <AutoLogin>
    <RouterProvider router={router}/>
  </AutoLogin>
</Provider>
```

COPY

← PREV

NEXT →